Actividad práctica en parejas

Objetivo: tomar un juego que guarda puntajes en SQLite con todo el acceso a datos adentro del modelo, una conexión por partida y una prueba que solo corre contra la base real, y separarlo con DAO y Singleton. Después, agregar una consulta nueva y ver cuánto cuesta.

Los archivos

El código de la actividad está en el repositorio de la cátedra. Si ya está clonado, alcanza con actualizarlo desde la carpeta del repositorio:

```
git pull
```

Si no, se clona una única vez:

```
git clone https://github.com/POO-CR/poo-2026.git
```

También está en archivo comprimido en el classroom. La carpeta de hoy es *16-persistencia/actividad*.

El proyecto necesita el driver de SQLite, *sqlite-jdbc-3.50.3.0.jar*, en la carpeta *lib*. En el archivo comprimido del classroom ya viene. El repositorio no guarda archivos *.jar*, así que quien lo use desde ahí tiene que copiarlo a *lib* como indica la pre clase.

Puntaje es un puntaje de la tabla de honor, y no se modifica. *Partida* suma puntos y, al terminar, guarda el puntaje. Para eso abre su propia conexión a *juego.db*, crea la tabla y escribe el SQL. *Conexion*, en el paquete *persistencia*, está vacía. *Programa* juega una partida de Ana y otra de Luis y muestra la tabla de honor. *PruebaPartida* es otro programa con su propio *main*, una prueba de *Partida*.

Parte 1: ver el problema

1. Ejecutar *Programa*. Aparece el archivo *juego.db* en la carpeta del proyecto, la tabla de honor tiene dos puntajes, y la última línea dice que se abrieron dos conexiones. Ejecutar de nuevo y ver que los puntajes se acumulan.
2. Mirar los *import* de *Partida*. Contar cuántas clases de *java.sql* usa una clase del modelo.
3. Ejecutar *PruebaPartida* dos veces y después *Programa*. La tabla de honor del juego ahora tiene al jugador “Prueba” en el primer lugar, dos veces. La prueba escribió en la base de verdad, porque *Partida* no tiene forma de trabajar con otra.
4. Borrar *juego.db* antes de seguir, para empezar con la tabla limpia.

Parte 2: Singleton y DAO

Un TODO por vez, en orden. Cada TODO indica en qué archivos se trabaja. Entre el TODO 2 y el TODO 4 el proyecto no compila: es esperable.

- TODO 1 (*Conexion.java*): *Conexion* pasa a ser Singleton. El constructor privado abre la conexión con *DriverManager.getConnection("jdbc:sqlite:juego.db")* y suma uno al contador. Un atributo estático guarda la única instancia, *getInstancia()* la crea la primera vez, y *get()* devuelve la *Connection*. Los dos métodos pueden declarar *throws SQLException*.
- TODO 2 (clase nueva en *modelo*): la interfaz *PuntajeDAO*, con *boolean guardar(Puntaje puntaje)* y *List mejores(int cantidad)*. Sin una palabra de SQL, y en el paquete *modelo*, porque la define lo que la partida necesita.
- TODO 3 (clase nueva en *persistencia*): *PuntajeDAOSQLite* implementa *PuntajeDAO*. Recibe la *Connection* por constructor, crea la tabla si no existe, y sus dos métodos tienen el SQL que hoy está en *terminar()* y en *tablaDeHonor()* de *Partida*, adaptado para recibir el *Puntaje* y la cantidad.
- TODO 4 (*Partida.java*, *Programa.java*, *PruebaPartida.java* y una clase nueva): *Partida* recibe el jugador y un *PuntajeDAO* por constructor. *terminar()* le pide al DAO que guarde un *Puntaje* nuevo, *tablaDeHonor()* le pide los cinco mejores, y el contador de conexiones, los *import* de *java.sql* y todo el SQL desaparecen. En *persistencia*, *PuntajeDAOMemoria* implementa la misma interfaz con una lista. *Programa* obtiene la conexión una sola vez con *Conexion.getInstancia().get()*, crea un único *PuntajeDAOSQLite* y se lo pasa a las dos partidas, y al final muestra *Conexion.getConexionesAbiertas()*. *PruebaPartida* crea su partida con *new PuntajeDAOMemoria()*.

Con los cuatro TODO hechos, *Programa* muestra la misma tabla de honor con una sola conexión abierta. Ejecutar *PruebaPartida* varias veces y después *Programa*, y comprobar que "Prueba" ya no aparece en la tabla.

Parte 3: una consulta más

- TODO 5: la pantalla de estadísticas quiere los mejores puntajes de un jugador. Agregar a *PuntajeDAO* el método *List mejoresDe(String jugador, int cantidad)* e implementarlo en las dos clases. En *Programa*, mostrar los tres mejores puntajes de Ana.

Antes de seguir, comprobar que ninguna clase del paquete *modelo* importa *java.sql*, y que *getInstancia()* aparece en un solo lugar de todo el proyecto.

Para discutir en la pareja y anotar:

- El TODO 5 obligó a tocar tres archivos para una consulta. ¿Es un costo del patrón o un defecto de este diseño? ¿Qué pasaría con diez consultas?
- *PuntajeDAOSQLite* recibe la conexión por constructor en vez de llamar a *Conexion.getInstancia()*. ¿Qué permite eso? SQLite acepta la URL *jdbc:sqlite::memory:*, que crea una base que desaparece al cerrar el programa. ¿Cómo se usaría para probar *PuntajeDAOSQLite* sin tocar *juego.db*?
- *PuntajeDAOSQLite* atrapa la *SQLException* y devuelve *false* o una lista vacía. ¿Qué gana el modelo con eso? ¿Qué se pierde?

- Si el nombre del jugador se pegara con + adentro del texto del *INSERT* en vez de ir en un hueco, ¿qué pasaría con un jugador llamado *x', 0, ''); DROP TABLE puntajes; –*?
- En el proyecto integrador, ¿qué entidades se guardan? ¿Qué operaciones necesita el modelo de cada una? ¿Cuál es la única clase que debería llamar a *getInstancia()*?